

Recap

We used a decoder to predict and generate tokens.

TRANSFORMERS AND PRETRAINING

Input

Combine token embeddings with position information.

Transformer blocks

Use causal attention and feedforward layers to form context vectors.
Each block also uses residual connections and layer normalization.

Language model head

Apply unembedding and softmax to give next-token probabilities.

Pretraining

Use the next tokens in the text as targets.
Train the model with cross-entropy loss.

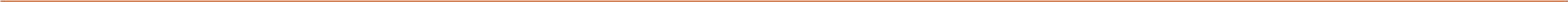
Decoding

Add the chosen token to the input. Repeat the process.
Temperature, top-k, and top-p control token choice.

We can predict the next token. How can we use this model for other language tasks?

Foundation Models and Adaptation

We will see how pretrained models can be used for different tasks.



MODULE 4 • THE OPENING QUESTION

We made a model that predicts the next token.
How can it classify sentiment, translate text, or give a response to a question?

Task	Context for the model		Response
Sentiment	Review: “The film was wonderful.” Sentiment:	→	Positive
Translation	English: Good morning French:	→	Bonjour
Question answering	Q: What is the capital of Japan? A:	→	Tokyo

The model gives each response with the same next-token prediction loop.
These examples use the decoder model from the previous classes.

Which patterns must a model learn during pretraining to do these different tasks?

Chapter 9: Masked Language Models

We study masked language models in the order used in Chapter 9.

CHAPTER OUTLINE

Three architectures for language models

Bidirectional Transformer Encoders

Training Bidirectional Encoders

Fine-Tuning for Classification

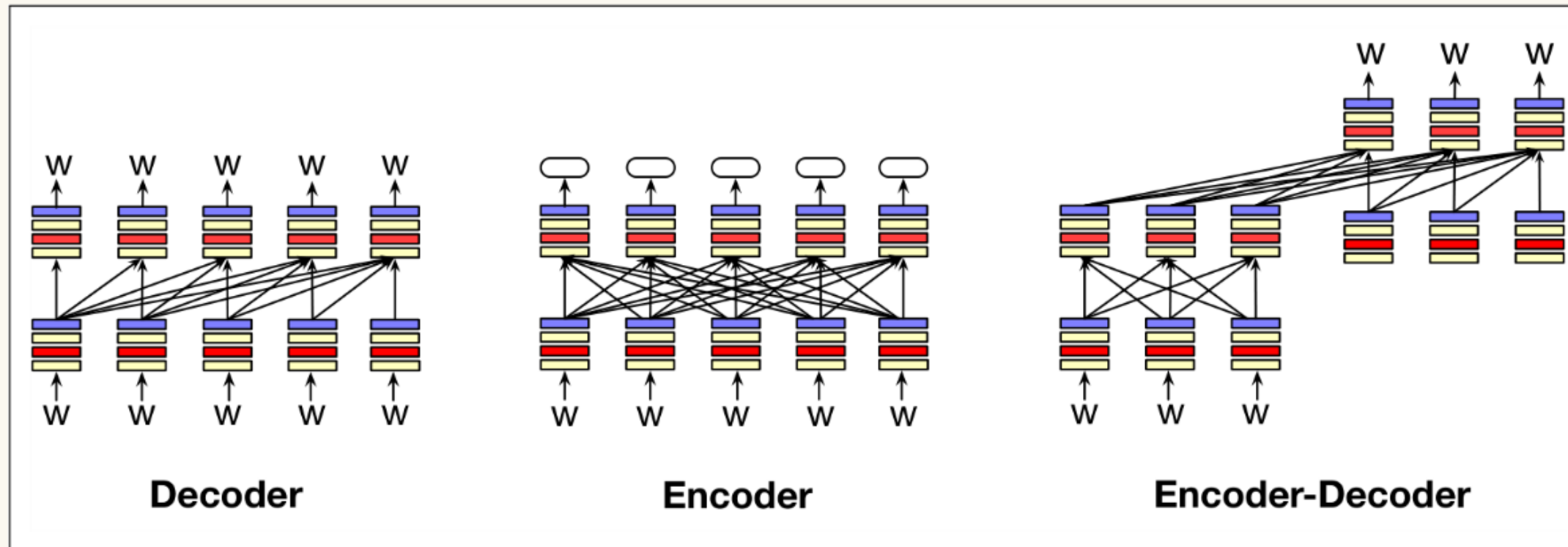
Fine-Tuning for Sequence Labeling:
Named Entity Recognition

How do the three architectures use input and output tokens?

Three Language Model Architectures

A Transformer can encode input tokens, generate output tokens, or do both.

BERT = BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS



Decoder (GPT)

Use prior tokens.
Generate one token at a time.
Example: text generation.

Encoder (BERT)

Use context on both sides.
Give one vector per input token.
Use a classifier for task labels.

Encoder-decoder (T5)

Encode an input sequence.
Generate an output sequence.
Input and output lengths can differ.
Example: translation.

What changes when a token can use input tokens on both sides?

Language Model Architectures: Key Differences

Model architecture determines how a model uses inputs and produces outputs.

MODEL EXAMPLES AND TYPICAL USES

- GPT, Claude, Llama, and Mistral use decoder architectures. They predict the next token from the preceding tokens.
- BERT and RoBERTa predict masked tokens. Their training uses context on both sides.
- BERT-type encoders do not generate text one token at a time. We can add a classifier and fine-tune the model on labeled data.
- Decoder and encoder–decoder models are generative models. They produce output tokens from an input sequence.
- Encoder–decoder inputs and outputs can have different lengths or token types. Examples include translation and speech recognition.

How does a bidirectional encoder represent each input token?

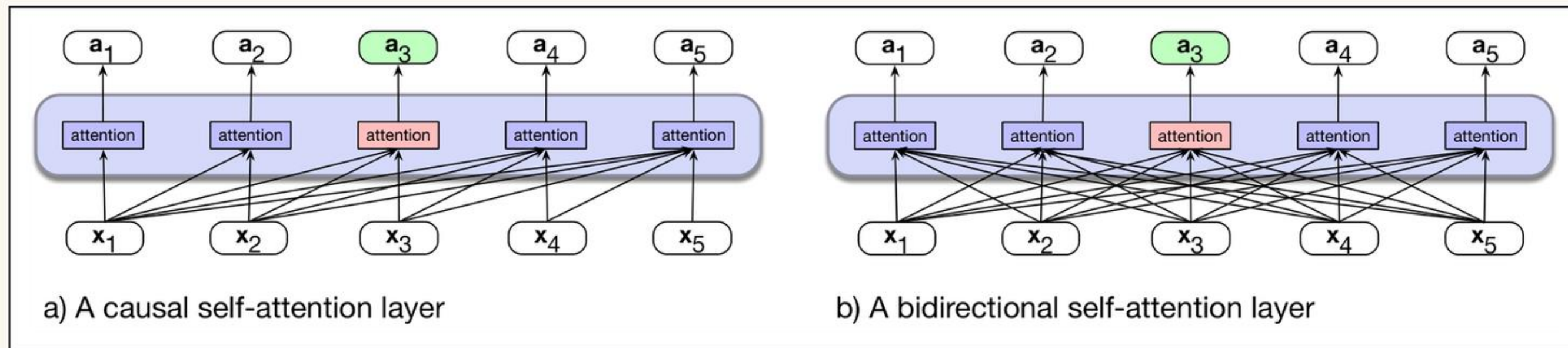
Bidirectional Context

The full input can help us interpret each token.

INFORMATION AVAILABLE AT POSITION 3

Causal attention

Bidirectional attention



Use positions 1 to 3.

Use positions 1 to 5.

“She sat on the bank of the river.”

Later words can help identify what “bank” means.

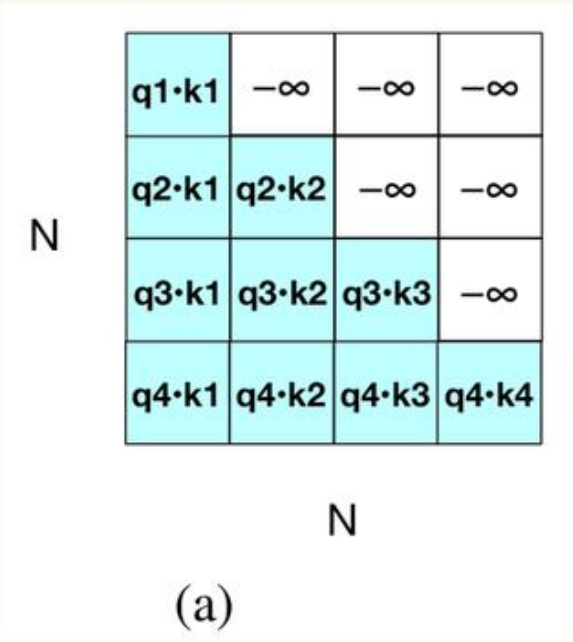
How do we allow attention to use later input positions?

Attention Without the Causal Mask

Bidirectional attention can use every valid input position.

CAUSAL AND BIDIRECTIONAL ATTENTION

Causal attention

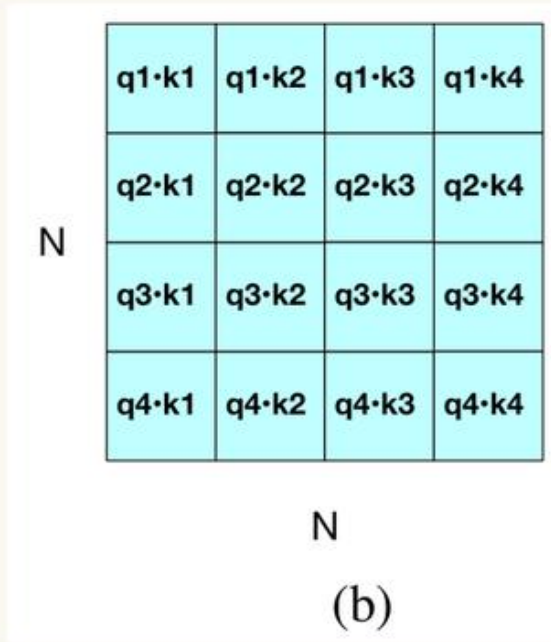


$$\text{head} = \text{softmax}\left(\text{mask}\left(\frac{QK^T}{\sqrt{d_k}}\right)\right) V$$

Later-token attention weights are zero.

The query, key, and value computations keep the same roles.

Bidirectional attention



$$\text{head} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

All valid input positions can contribute.

What input does this encoder use, and what does it return?

Encoder Inputs and Contextual Outputs

The encoder gives a contextual vector for each input token.

SUBWORD INPUTS AND CONTEXTUAL VECTORS

$$X \in \mathbb{R}^{N \times d} \xrightarrow{\text{bidirectional encoder}} H \in \mathbb{R}^{N \times d}$$

Input embeddings (X)

Tokenize text with BPE, WordPiece, or SentencePiece Unigram LM.

Combine token embeddings with position information.

Original BERT adds segment embeddings to mark which text each token belongs to.

Transformer blocks

Use multihead attention, feedforward layers, residual connections, and layer normalization.

Contextual outputs (H)

Keep one output vector for each input token.

Each vector can use information from the full input sequence.

N : number of input tokens d : model width

What configurations do BERT and XLM-RoBERTa use?

BERT and XLM-RoBERTa Configurations

BERT-base uses English training data. XLM-RoBERTa uses data from 100 languages.

MODEL CONFIGURATIONS

Setting	BERT-base	XLM-RoBERTa (large)
Language coverage	English	100 languages
Subword tokenizer	WordPiece	SentencePiece Unigram LM
Vocabulary size	30,000 tokens	250,000 tokens
Maximum context length	512 tokens	512 tokens
Model width	768	1024
Input shape at the limit	512×768	512×1024
Transformer blocks	12	24
Attention heads per block	12	16
Parameters	≈ 100 million	≈ 550 million

N varies by input. 512 is the limit. Llama 3 includes a model with 405 billion parameters.

How can we train an encoder to predict tokens inside its input?

BERT's Token Selection and Replacement

Pretraining learns from a large text corpus. Its prediction targets come from the text.

ORIGINAL BERT • INPUT CHANGES FOR PRETRAINING

Select 15% of token positions for prediction.

Original token at one selected position: slept

Among the selected positions

80%

Use the special token [MASK].

[MASK]

Target: slept

10%

Replace with a random token.

table

Target: slept

10%

Keep the original token.

slept

Target: slept

Predict the original token in all three cases.

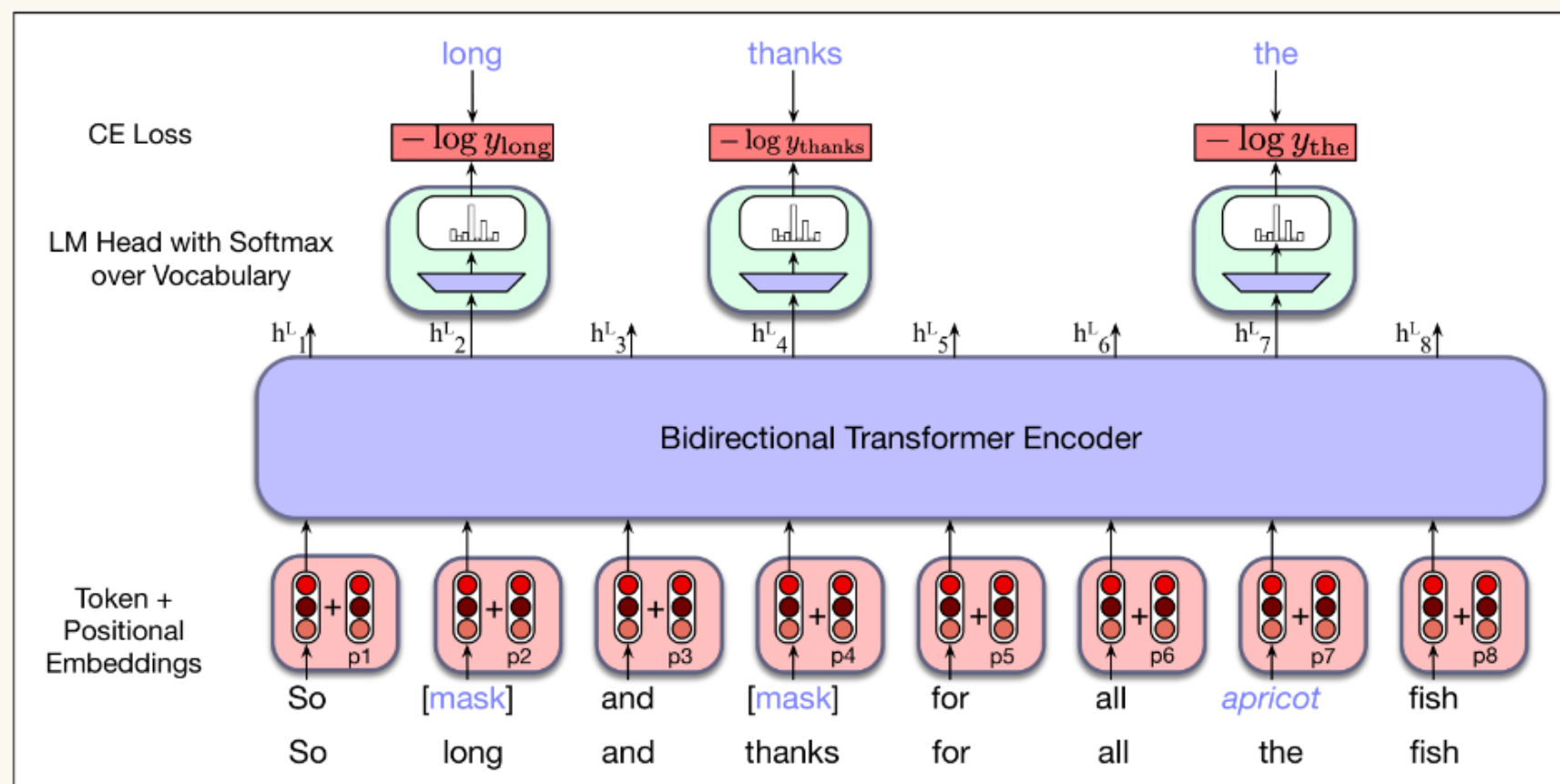
This reduces dependence on [MASK], which is absent from ordinary task inputs.

How do we predict the original tokens from the changed input?

Masked Language Model Training

MLM = masked language modelling. Predict the original tokens at selected positions.

CHANGED INPUT, VOCABULARY PREDICTIONS AND LOSS



Vocabulary prediction head

$$\mathbf{u}_i = \mathbf{h}_i^L E^\top$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

$\mathbf{h}$: final encoder vector.

i : token position. L : last layer.

E : token embedding matrix.

$\mathbf{u}$: scores. $\mathbf{y}$: probabilities.

Compare with the original token.

Only selected positions supply loss terms.

A head maps encoder vectors to output scores. Softmax gives probabilities.

What loss do we obtain from one selected position?

An MLM Training Example

Predict the original token, calculate the loss, then update the parameters.

ONE SELECTED POSITION • SUPPLIED PROBABILITY

Original lunch was delicious

Input lunch was [MASK]

Target delicious at position 3

Assume each word is one token.

M: selected positions. |M|: their count.

The tilde marks the input after replacement.

$$P(x_3 = \text{delicious} \mid \tilde{\mathbf{x}}) = 0.25$$

$$\ell_3 = -\ln(0.25) \approx 1.386 \text{ nats}$$

$$L_{\text{MLM}} = -\frac{1}{|M|} \sum_{i \in M} \ln P(x_i \mid \tilde{\mathbf{x}}), \quad M = \{3\}$$

Average loss over selected positions.

One gradient-descent step

Update the embeddings, encoder blocks,
and vocabulary head.

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L_{\text{MLM}}$$

θ : model parameters. η : learning rate.

Context tokens can receive gradients even though their positions have no loss term.

How can pretraining also use pairs of sentences?

Applications of Sentence-Pair Models

Compare two texts to answer a practical question.

APPLICATION TASKS

Duplicate questions

Paraphrase detection

A: Where are the lecture videos?
B: How can I watch previous lectures?
Can we use the same FAQ answer?

Same meaning

Check a claim

Entailment

A: Meera submitted the assignment early.
B: Meera submitted the assignment.
Does A support the claim in B?

Supported claim

Check paragraph flow

Discourse coherence

A: We heated the sample.
B: Its electrical resistance decreased.
Do the sentences form connected text?

Connected text

NSP pretraining was intended to help BERT learn representations of text pairs.

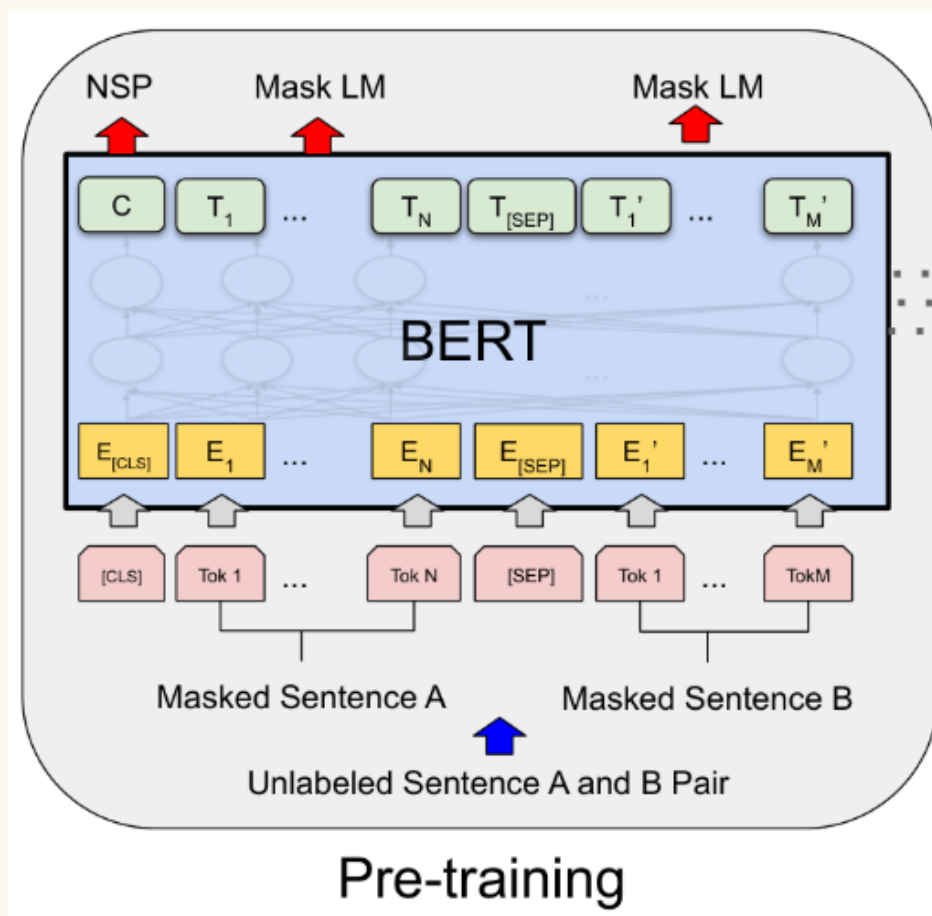
Each application then needs training with its own task labels.

How can document order supply labels for a pretraining task?

Next Sentence Prediction

Predict whether the second text segment follows the first in the source document.

ORIGINAL BERT • THE SECOND PRETRAINING TASK



[CLS]: special start token. Its final vector feeds the classifier.

[SEP]: special token that ends a text segment.

Pair input: [CLS] A [SEP] B [SEP]

Segment A: The cat slept on the mat.

50%: Use the actual following segment.

It woke when the door opened. (IsNext)

50%: Use a random segment.

The train leaves at noon. (NotNext)

The NSP classifier uses the final [CLS] vector.

Use cross-entropy for the NSP label.

Both losses update the shared encoder and their respective prediction heads.

Do all encoders use the same pretraining objectives?

Training Regimes

Encoder models differ in their training data and objectives.

BRIEF OVERVIEW

Original BERT

Train on pairs of text segments with MLM and NSP.
Use English Wikipedia and BooksCorpus.

RoBERTa

Train on consecutive text with MLM. Omit NSP.
Use more training data.

Multilingual encoders

Train on text in multiple languages.
Give languages with less data more weight when learning the vocabulary.

Details for self-study: J&M §9.3.3, pp.234–235

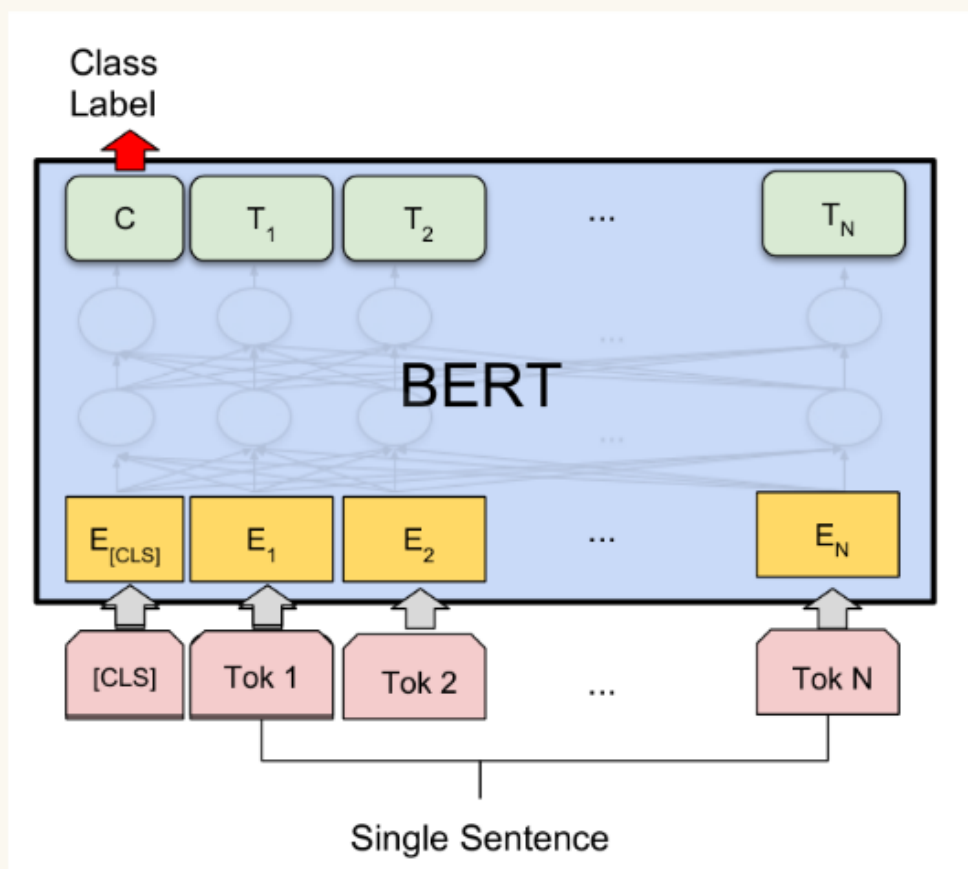
Training data and settings. Multilingual sampling, Eq. 9.5.
Benefits and limits of multilingual models.

How can we use the pretrained encoder to classify a complete text?

Sequence Classification

A classifier head maps the final [CLS] vector to probabilities over task labels.

§9.4.1 • ONE TEXT → ONE LABEL



Example: sentiment classification

The film was wonderful.

[CLS] The film was wonderful. [SEP]

Use the final [CLS] vector (C in the figure).

$$\mathbf{y} = \text{softmax}(\mathbf{h}_{\text{CLS}}^L W_C)$$

h: final [CLS] vector. L: last layer.

W: classifier weights. y: class probabilities.

Labels: Positive, Negative, Neutral. Target: Positive.

Train with labelled texts and cross-entropy.

At inference, all parameters stay fixed.

Frozen encoder: train the head while encoder weights stay fixed.

Fine-tuned encoder: update the head and some or all encoder weights.

What changes when a label depends on a pair of texts?